

Experiment 2

Module Name: Python BDD Restful Automations

Experiment Title: Use of test data driven Automation in Python Behave Framework

Problem statement: Writing a separate Gherkin scenario for every possible login combination (valid, invalid, locked out) creates repetitive and bloated feature files.

Objective: Perform Python automations using Python Behave Framework

Tools, software, and concepts used: Python 3, Behave BDD framework, Gherkin Scenario Outline, Gherkin Examples tables, Parameterization.

Source code or implementation steps:

1. Create any Folder
2. Inside that folder create 'features' Folder
3. Inside that 'features' folder create 'steps' folder and a 'data_driven_login.feature' file
4. Inside that 'data_driven_login.feature' file write this:

Feature: Data-Driven Login Validation

Scenario Outline: Verify login behavior with multiple credential sets

Given the user opens the login portal

When the user enters username "<username>" and password "<password>"

And clicks the login submit button

Then the outcome should match the expected "<status>"

Examples:

username	password	status	
standard_user	secret_sauce	success	
locked_out_user	secret_sauce	locked	
invalid_user	wrong_pass	failure	

5. Inside 'steps' folder create 'data_driven_steps.py' and write this

```
from behave import when, then
from selenium.webdriver.common.by import By

@when('the user enters "{username}" and "{password}"')
```

```

def step_impl(context, username, password):
    context.driver.find_element(By.ID, "user-name").send_keys(username)
    context.driver.find_element(By.ID, "password").send_keys(password)

@then('the login outcome should be "{expected_result}")
def step_impl(context, expected_result):
    if expected_result == "success":
        assert "inventory.html" in context.driver.current_url
    elif expected_result == "locked_out":
        error = context.driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
        assert "locked out" in error
    elif expected_result == "invalid":
        error = context.driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
        assert "Username and password do not match" in error
    context.driver.quit()

```

6. In the folder go to cmd and run this command

```
>>> python -m behave features/data_driven_login.feature
```

Output with clearly labelled screenshots:

```

F:\My Desktop\Project>python -m behave features/data_driven_login.feature
USING RUNNER: behave.runner:Runner
Feature: Data-Driven Login Validation # features/data_driven_login.feature:1

Scenario Outline: Verify login behavior with multiple credential sets -- @1.1
  Given the user opens the login portal
  When the user enters username "standard_user" and password "secret_sauce"
  And clicks the login submit button
  Then the outcome should match the expected "success"
# features/data_driven_login.feature:11
# features/steps/data_driven_steps.py:7 4.508s
# features/steps/data_driven_steps.py:13 0.210s
# features/steps/data_driven_steps.py:25 0.093s
# features/steps/data_driven_steps.py:29 2.479s

Scenario Outline: Verify login behavior with multiple credential sets -- @1.2
  Given the user opens the login portal
  When the user enters username "locked_out_user" and password "secret_sauce"
  And clicks the login submit button
  Then the outcome should match the expected "locked"
# features/data_driven_login.feature:12
# features/steps/data_driven_steps.py:7 1.637s
# features/steps/data_driven_steps.py:13 0.366s
# features/steps/data_driven_steps.py:25 0.110s
# features/steps/data_driven_steps.py:29 2.577s

Scenario Outline: Verify login behavior with multiple credential sets -- @1.3
  Given the user opens the login portal
  When the user enters username "invalid_user" and password "wrong_pass"
  And clicks the login submit button
  Then the outcome should match the expected "failure"
# features/data_driven_login.feature:13
# features/steps/data_driven_steps.py:7 1.653s
# features/steps/data_driven_steps.py:13 0.253s
# features/steps/data_driven_steps.py:25 0.091s
# features/steps/data_driven_steps.py:29 2.813s

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
12 steps passed, 0 failed, 0 skipped
Took 0min 16.791s

```

Screenshot 1: Terminal output showing the execution of the Scenario Outline iterating 3 times]

Console Output:

1 feature passed, 0 failed, 0 skipped

3 scenarios passed, 0 failed, 0 skipped

12 steps passed, 0 failed, 0 skipped

Took 0min 14.559s

Brief result: The Behave framework automatically executed the same scenario three times, injecting a new row of data from the Examples table during each iteration.

Observation: By wrapping variables in angle brackets < > in the feature file and matching them with curly braces { } in the Python step definitions, Behave handles variable parameterization natively.

Conclusion: Using Scenario Outline is the standard BDD approach for data-driven testing (DDT). It drastically reduces code duplication and makes expanding test coverage as simple as adding a new line to a text table.